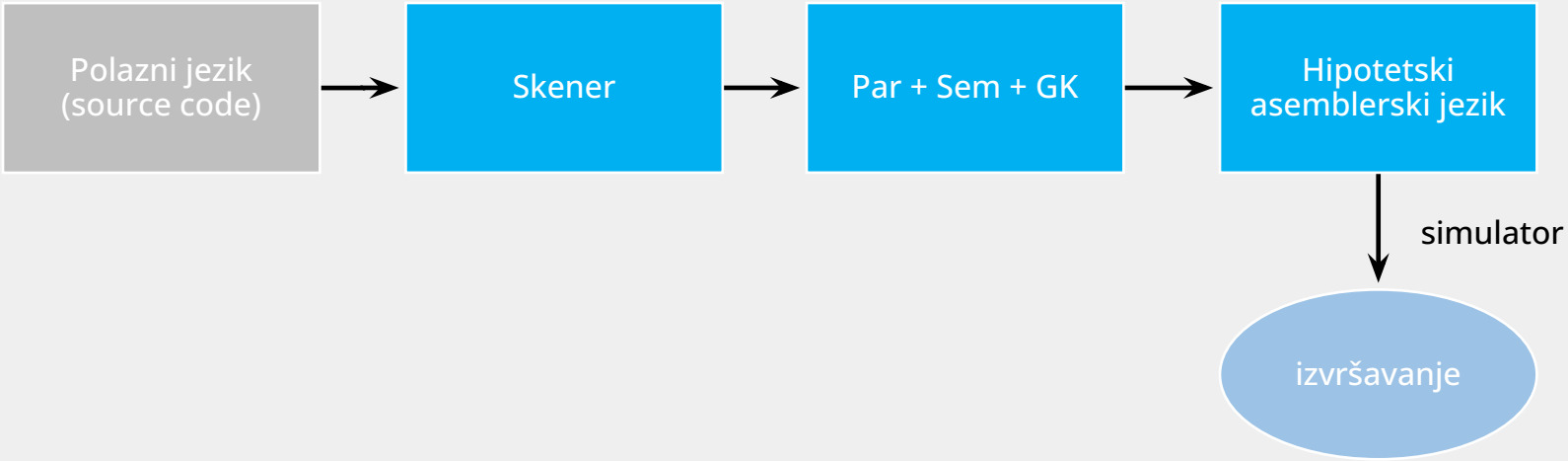
Programski prevodioci

08 Generisanje koda - micko

25-26

Dunja Vrbaški

Fakultet tehničkih nauka, Novi Sad

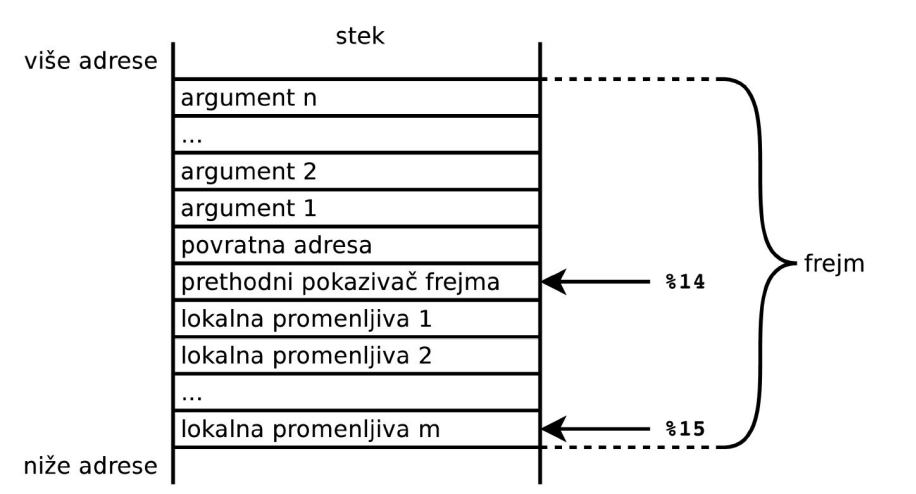

Polazni jezik
(source code)
Skener
Par + Sem + GK
Hipotetski
asemblerski jezik
simulator
izvršavanje

Hipotetski asemblerski jezik i izvršno okruženje

- hipotetska reigstarska mašina
- registri i memorijske lokacije zauzimaju 4 bajta
- postoji 16 registara

  - %0...%12 – registri opšte namene
  - %13 – povratna vrednost funkcije
  - %14 – pokazivač frejma
  - %15 – pokazivač steka

Stack frame

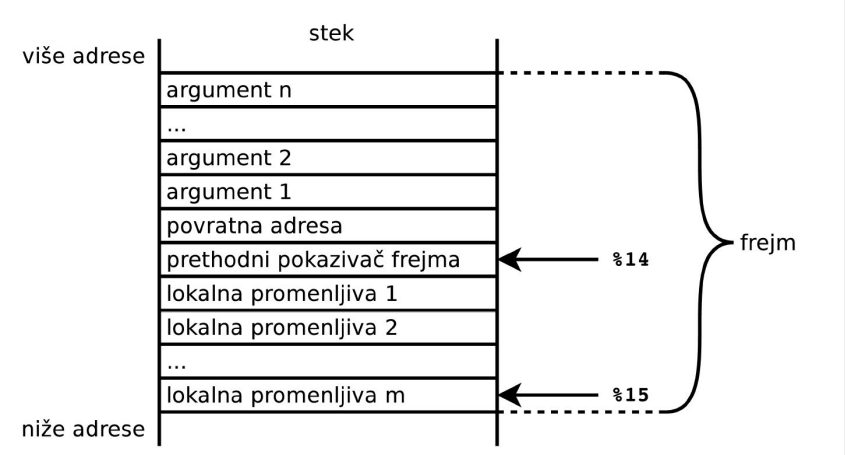


- %0...%12 – registri opšte namene
- %13 – povratna vrednost funkcije
- %14 – pokazivač frejma (FP)
- %15 – pokazivač steka (SP)

Argumenti

Adresiraju se u odnosu na FP (%14)

argument 1: 8(%14)
argument 2: 12(%14)
...
argument n: 4+n*4(%14)



indeksni operand: x(%y)
x je broj ili labela, y je registar

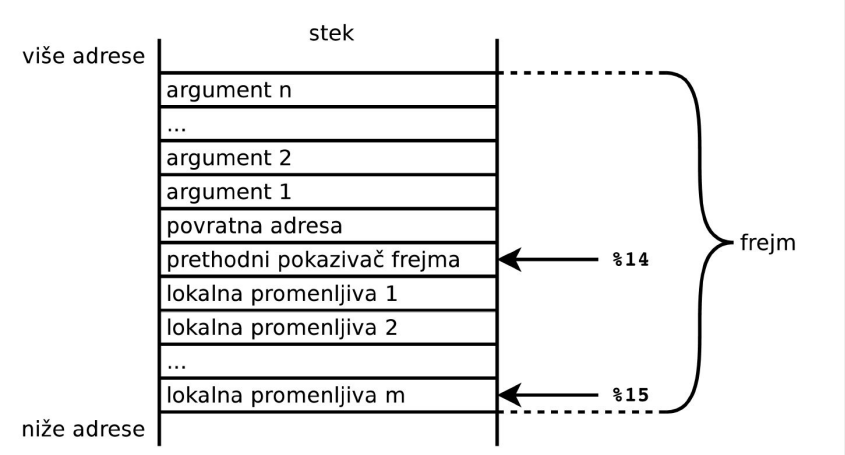
Vrednost: sadržaj memorijske lokacije koju adresira
- zbir vrednosti broja i sadržaja registra
- zbir adrese labele i sadržaja registra

Primeri: 8(%14), -8(%14), label1(%0),...

Lokalne promenljive

Adresiraju se u odnosu na FP (%14)

lokalna promenljiva 1: `-4(%14)`
lokalna promenljiva 2: `-8(%14)`
...
lokalna promenljiva n: `-m*4(%14)`



Instrukcije

```
CMPx ulazni operand, ulazni operand
```

naredba poređenja – postavlja status registra u skladu sa rezultatom
x: S - označeni, U - neoznačeni

```
JMP   ulazni operand
```

naredba bezuslovnog skoka – smešta vrednost operanda u programski brojač

```
JEQ  ulazni operand
JNE  ulazni operand
JGTx ulazni operand
JLTx ulazni operand
JGEx ulazni operand
JLEx ulazni operand
```

naredbe uslovnog skoka – smešta vrednost operanda u programski brojač u zavisnosti od sadržaja status registra

```
PUSH    ulazni operand
POP     izlazni operand
```

naredbe upravljanja stekom

- smeštanje vrednosti operanda na stek (push)
- preuzimanje vrednosti sa steka i smeštanje u operand (pop)

```
CALL ulazni operand
```

naredba poziva potprograma

- smeštanje sadržaja programskog brojača na stek i
- smeštanje vrednosti ulaznog operanda u programski brojač

```
RET
```

naredba povratka iz potprograma

- preuzimanje vrednosti sa vrha steka i smeštanje te vrednosti u programski brojač

```
ADDx ulazni operand, ulazni operand, izlazni operand
SUBx ulazni operand, ulazni operand, izlazni operand
```

naredbe za aritmetičke operacije

```
MULx ulazni operand, ulazni operand, izlazni operand
DIVx ulazni operand, ulazni operand, izlazni operand
```

```
MOVx ulazni operand, izlazni operand
```

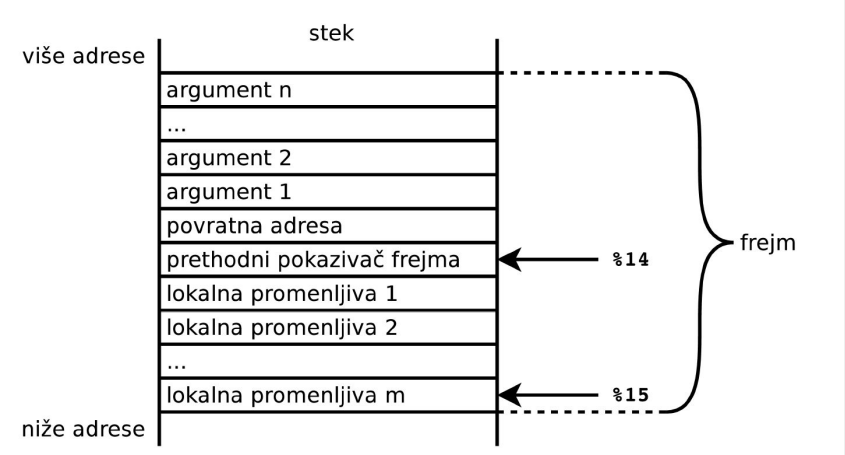
naredba za prebacivanje vrednosti

```
WORD broj
```

direktiva za zauzimanje određenog broja memorijskih lokacija

Primer prevođenja

```
int fun() {
   int a;
   int b;
   a = 3;
   b = 2;
   a = b - a;
   return a;
}
```



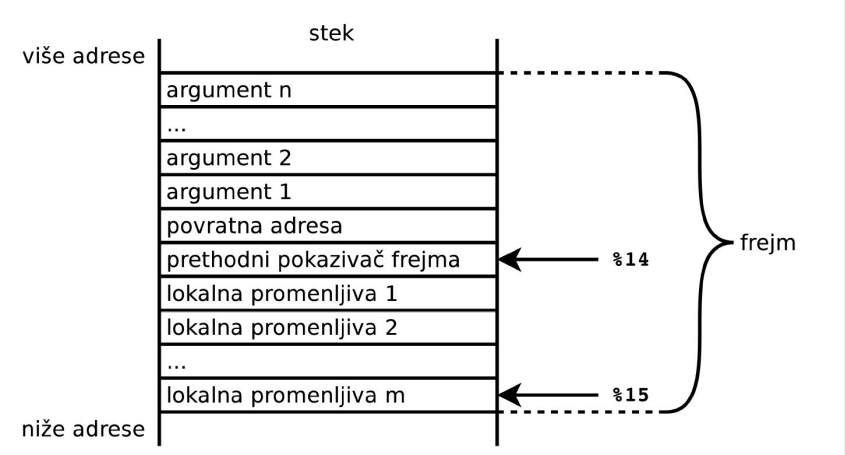

```
SUBS      -8(%14), -4(%14), %0
MOV       %0, -4(%14)
```

- međurezultat smeštamo u registar
- pretpostavili smo da su svi radni registri slobodni

Primer prevođenja

```
a = (a + b) + (c + d) – e;
```

```
ADDS      -4(%14),-8(%14),%0
ADDS      -12(%14),-16(%14),%1
ADDS      %0,%1,%0
SUBS      %0,-20(%14),%0
MOV       %0,-4(%14)
```



- međurezultat smeštamo u registre
- pretpostavili smo da su svi radni registri slobodni
- pretpostavili smo da su: a, b, c, d i e lokalne promenljive neke funkcije deklarisane tim redom

```
ADDS      -4(%14),-8(%14),%0
ADDS      -12(%14),-16(%14),%1
ADDS      %0,%1,%0
SUBS      %0,-20(%14),%0
MOV       %0,-4(%14)
```

Prilikom implementacije moraćemo da:

- utvrdimo i ispišemo odgovarajuće instrukcije (ADDS, SUBS, MOV)
  razmišljanje + pomoćne funkcije za ispis

- odredimo operande instrukcija, koeficijente za indeksne operande...
  razmišljanje + TS

- odredimo slobodne registre
  TS

Postupak

Izlaz → fajl sa kodom napisanim u hipotetskom asemblerskom jeziku

Generišemo fajl i, dok parsiramo, popunjavamo instrukcijama

```
assignment_statement
  : _ID _ASSIGN num_exp _SEMICOLON
  ;
```

```
assignment_statement
  : _ID _ASSIGN num_exp _SEMICOLON
      {
        int idx = lookup_symbol($1, VAR|PAR);
        if(idx == NO_INDEX)
          err("invalid lvalue '%s' in assignment", $1);
        else
          if(get_type(idx) != get_type($3))
            err("incompatible types in assignment");
      }
  ;
```

```
assignment_statement
  : _ID _ASSIGN num_exp _SEMICOLON
      {
        int idx = lookup_symbol($1, VAR|PAR);
        if(idx == NO_INDEX)
          err("invalid lvalue '%s' in assignment", $1);
        else
          if(get_type(idx) != get_type($3))
            err("incompatible types in assignment");
        ... GENERISANJE KODA...
      }
  ;
```

micko fajlovi:

defs.h	*dopunjen*
codegen.h	*dodat*
codegen.c	*dodat*
micko.y	*dopunjen*

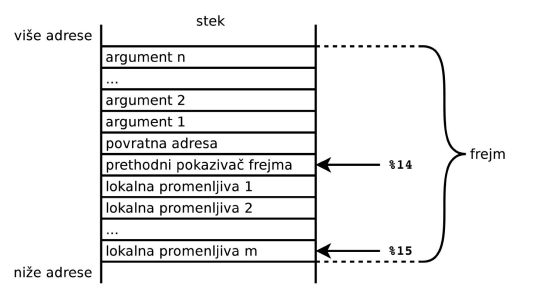
Pre implementacije → Pregled generisanog koda za poziv i definiciju funkcije

```
a = f(a + b);
```

```
ADDS    -4(%14),-8(%14),%0      1
PUSH    %0                      2
CALL    f                       3
ADDU    %15,$4,%15              4
MOV     %13,-4(%14)             5
```

CALL ulazni operand

naredba poziva potprograma
- smeštanje sadržaja programskog brojača na stek i
- smeštanje vrednosti ulaznog operanda u programski brojač

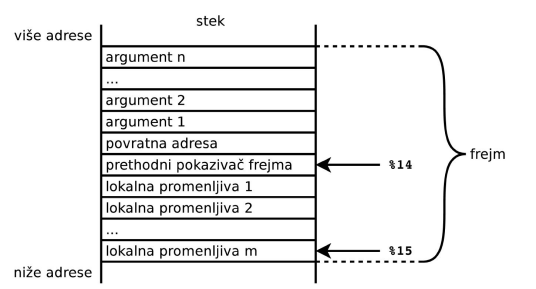


```
int f(int p) {
   int a;
   return p + a;
}
```

```
f:                                  1
   PUSH   %14                       2
   MOV    %15,%14                   3
   SUBS   %15,$4,%15                4
@f_body:                            5
   ADDS   8(%14),-4(%14),%0         6
   MOV    %0,%13                    7
   JMP    @f_exit                   8
@f_exit:                            9
   MOV    %14,%15                   10
   POP    %14                       11
   RET                              12
```

RET

- naredba povratka iz potprograma
- preuzimanje vrednosti sa vrha steka i smeštanje te vrednosti u programski brojač



```c
int f(int p) {
   int a;
   a = 3;
   return p + a;
}

int main() {
   int a;
   a = f(7);
}
```

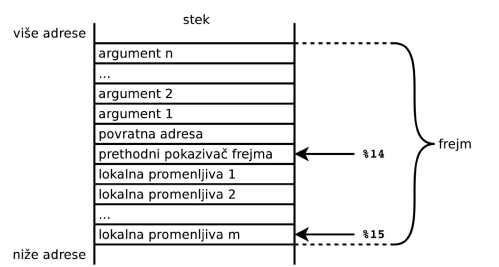
```
f:                                  1
   PUSH   %14                       2
   MOV    %15,%14                   3
   SUBS   %15,$4,%15                4
@f_body:                            5
   MOV    $3,-4(%14)                6
   ADDS   8(%14),-4(%14),%0         7
   MOV    %0,%13                    8
   JMP    @f_exit                   9
@f_exit:                            10
   MOV    %14,%15                   11
   POP    %14                       12
   RET                              13
…
   PUSH   $7                        21
   CALL   f                         22
   ADDS   %15,$4,%15                23
   MOV    %13,-4(%14)               24
```

```
int f(int p) {
   int a;
   a = 3;
   return p + a;
}

int main() {
   int a;
   a = f(7);
}
```

```
f:                                  1
   PUSH    %14                      2
   MOV     %15,%14                  3
   SUBS    %15,$4,%15               4
@f_body:                            5
   MOV     $3,-4(%14)               6
   ADDS    8(%14),-4(%14),%0        7
   MOV     %0,%13                   8
   JMP     @f_exit                  9
@f_exit:                            10
   MOV     %14,%15                  11
   POP     %14                      12
   RET                              13
...
   PUSH    $7                       21
   CALL    f                        22
   ADDS    %15,$4,%15               23
   MOV     %13,-4(%14)              24
```

7	SP



$13	RV
$14	FP
$15	SP

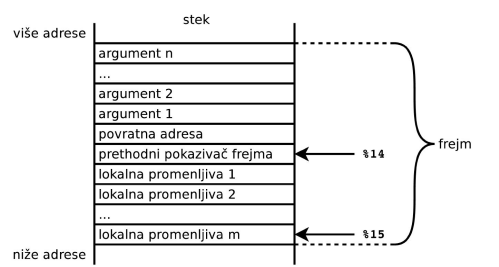
```c
int f(int p) {
    int a;
    a = 3;
    return p + a;
}

int main() {
    int a;
    a = f(7);
}
```

```
f:                                  1
    PUSH    %14                     2
    MOV     %15,%14                 3
    SUBS    %15,$4,%15              4
@f_body:                            5
    MOV     $3,-4(%14)              6
    ADDS    8(%14),-4(%14),%0       7
    MOV     %0,%13                  8
    JMP     @f_exit                 9
@f_exit:                            10
    MOV     %14,%15                 11
    POP     %14                     12
    RET                             13
...
    PUSH    $7                      21
    CALL    f                       22
    ADDS    %15,$4,%15              23
    MOV     %13,-4(%14)             24
```

7
addr 22

SP



$13	RV
$14	FP
$15	SP

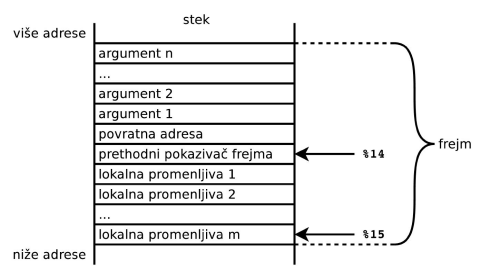
```
int f(int p) {
   int a;
   a = 3;
   return p + a;
}

int main() {
   int a;
   a = f(7);
}
```

```
f:                                  1
   PUSH    %14                      2
   MOV     %15,%14                  3
   SUBS    %15,$4,%15               4
@f_body:                            5
   MOV     $3,-4(%14)               6
   ADDS    8(%14),-4(%14),%0        7
   MOV     %0,%13                   8
   JMP     @f_exit                  9
@f_exit:                            10
   MOV     %14,%15                  11
   POP     %14                      12
   RET                              13
...
   PUSH    $7                       21
   CALL    f                        22
   ADDS    %15,$4,%15               23
   MOV     %13,-4(%14)              24
```

7
addr 22

SP



$13	RV
$14	FP
$15	SP

CALL ulazni operand

naredba poziva potprograma
- smeštanje sadržaja programskog brojača na stek i
- smeštanje vrednosti ulaznog operanda u programski brojač

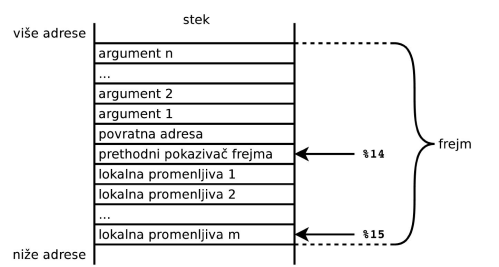
```c
int f(int p) {
   int a;
   a = 3;
   return p + a;
}

int main() {
   int a;
   a = f(7);
}
```

```
f:                                  1
   PUSH    %14                      2
   MOV     %15,%14                  3
   SUBS    %15,$4,%15               4
@f_body:                            5
   MOV     $3,-4(%14)               6
   ADDS    8(%14),-4(%14),%0        7
   MOV     %0,%13                   8
   JMP     @f_exit                  9
@f_exit:                            10
   MOV     %14,%15                  11
   POP     %14                      12
   RET                              13
...
   PUSH    $7                       21
   CALL    f                        22
   ADDS    %15,$4,%15               23
   MOV     %13,-4(%14)              24
```

7
addr 22
stari FP (iz %14)

SP



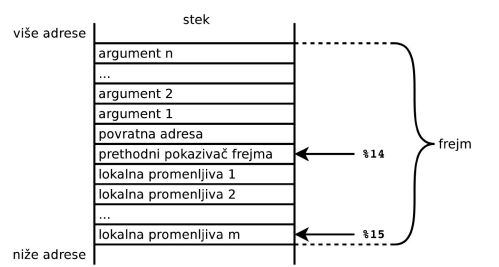
$13	RV
$14	FP
$15	SP

```c
int f(int p) {
   int a;
   a = 3;
   return p + a;
}

int main() {
   int a;
   a = f(7);
}
```

```
f:                                  1
   PUSH    %14                      2
   MOV     %15,%14                  3
   SUBS    %15,$4,%15               4
@f_body:                            5
   MOV     $3,-4(%14)               6
   ADDS    8(%14),-4(%14),%0        7
   MOV     %0,%13                   8
   JMP     @f_exit                  9
@f_exit:                            10
   MOV     %14,%15                  11
   POP     %14                      12
   RET                              13
...
   PUSH    $7                       21
   CALL    f                        22
   ADDS    %15,$4,%15               23
   MOV     %13,-4(%14)              24
```

7	
addr 22	
stari FP	SP, FP



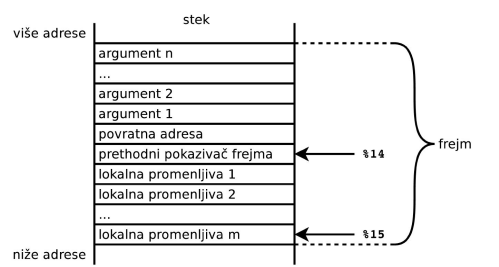
$13	RV
$14	FP
$15	SP

```
int f(int p) {
    int a;
    a = 3;
    return p + a;
}

int main() {
    int a;
    a = f(7);
}
```

```
f:                                  1
    PUSH    %14                     2
    MOV     %15,%14                 3
    SUBS    %15,$4,%15              4
@f_body:                            5
    MOV     $3,-4(%14)              6
    ADDS    8(%14),-4(%14),%0       7
    MOV     %0,%13                  8
    JMP     @f_exit                 9
@f_exit:                            10
    MOV     %14,%15                 11
    POP     %14                     12
    RET                             13
...
    PUSH    $7                      21
    CALL    f                       22
    ADDS    %15,$4,%15              23
    MOV     %13,-4(%14)             24
```

7	
addr 22	
stari FP	FP
mesto za l.p a	SP



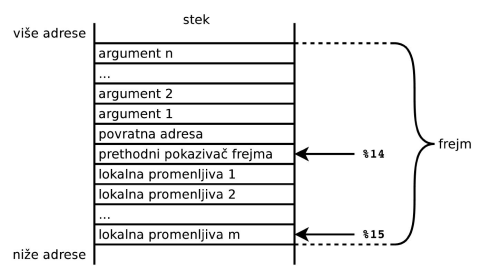
$13	RV
$14	FP
$15	SP

```c
int f(int p) {
    int a;
    a = 3;
    return p + a;
}

int main() {
    int a;
    a = f(7);
}
```

```
f:                                  1
    PUSH    %14                     2
    MOV     %15,%14                 3
    SUBS    %15,$4,%15              4
@f_body:                            5
    MOV     $3,-4(%14)              6
    ADDS    8(%14),-4(%14),%0       7
    MOV     %0,%13                  8
    JMP     @f_exit                 9
@f_exit:                            10
    MOV     %14,%15                 11
    POP     %14                     12
    RET                             13
...
    PUSH    $7                      21
    CALL    f                       22
    ADDS    %15,$4,%15              23
    MOV     %13,-4(%14)             24
```

7	
addr 22	
stari FP	FP
3	SP



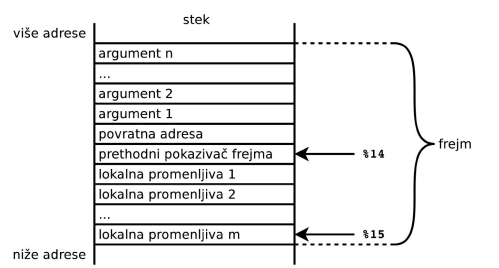
$13	RV
$14	FP
$15	SP

```c
int f(int p) {
    int a;
    a = 3;
    return p + a;
}

int main() {
    int a;
    a = f(7);
}
```

```
f:                                  1
    PUSH    %14                     2
    MOV     %15,%14                 3
    SUBS    %15,$4,%15              4
@f_body:                            5
    MOV     $3,-4(%14)              6
    ADDS    8(%14),-4(%14),%0       7
    MOV     %0,%13                  8
    JMP     @f_exit                 9
@f_exit:                            10
    MOV     %14,%15                 11
    POP     %14                     12
    RET                             13
...
    PUSH    $7                      21
    CALL    f                       22
    ADDS    %15,$4,%15              23
    MOV     %13,-4(%14)             24
```

7	
addr 22	
stari FP	FP
3	SP



$13	RV
$14	FP
$15	SP

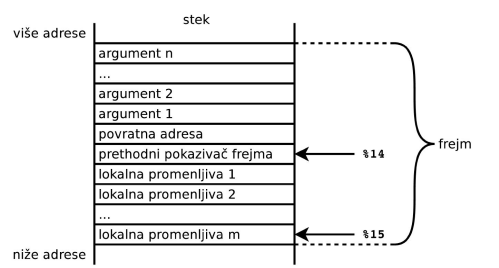
$0	10

```c
int f(int p) {
    int a;
    a = 3;
    return p + a;
}

int main() {
    int a;
    a = f(7);
}
```

```
f:                                  1
    PUSH    %14                     2
    MOV     %15,%14                 3
    SUBS    %15,$4,%15              4
@f_body:                            5
    MOV     $3,-4(%14)              6
    ADDS    8(%14),-4(%14),%0       7
    MOV     %0,%13                  8
    JMP     @f_exit                 9
@f_exit:                            10
    MOV     %14,%15                 11
    POP     %14                     12
    RET                             13
...
    PUSH    $7                      21
    CALL    f                       22
    ADDS    %15,$4,%15              23
    MOV     %13,-4(%14)             24
```

7	
addr 22	
stari FP	FP
3	SP



$13	10
$14	FP
$15	SP

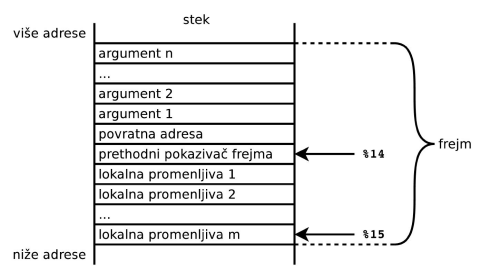
$0	10

```c
int f(int p) {
    int a;
    a = 3;
    return p + a;
}

int main() {
    int a;
    a = f(7);
}
```

```
f:                                  1
    PUSH    %14                     2
    MOV     %15,%14                 3
    SUBS    %15,$4,%15              4
@f_body:                            5
    MOV     $3,-4(%14)              6
    ADDS    8(%14),-4(%14),%0       7
    MOV     %0,%13                  8
    JMP     @f_exit                 9
@f_exit:                            10
    MOV     %14,%15                 11
    POP     %14                     12
    RET                             13
...
    PUSH    $7                      21
    CALL    f                       22
    ADDS    %15,$4,%15              23
    MOV     %13,-4(%14)             24
```

7	
addr 22	
stari FP	FP
3	SP



$13	10
$14	FP
$15	SP

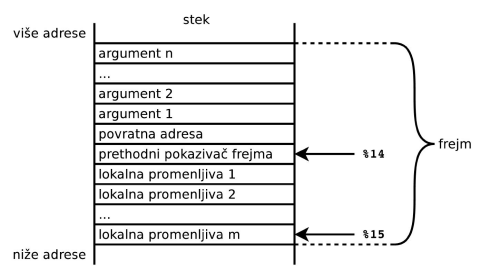
$0	~~10~~

```
int f(int p) {
    int a;
    a = 3;
    return p + a;
}

int main() {
    int a;
    a = f(7);
}
```

```
f:                                  1
    PUSH    %14                     2
    MOV     %15,%14                 3
    SUBS    %15,$4,%15              4
@f_body:                            5
    MOV     $3,-4(%14)              6
    ADDS    8(%14),-4(%14),%0       7
    MOV     %0,%13                  8
    JMP     @f_exit                 9
@f_exit:                            10
    MOV     %14,%15                 11
    POP     %14                     12
    RET                             13
...
    PUSH    $7                      21
    CALL    f                       22
    ADDS    %15,$4,%15              23
    MOV     %13,-4(%14)             24
```

7	
addr 22	
stari FP	FP, SP
3	~~SP~~



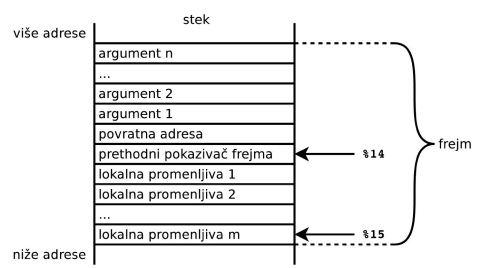
$13	10
$14	FP
$15	SP

```c
int f(int p) {
    int a;
    a = 3;
    return p + a;
}

int main() {
    int a;
    a = f(7);
}
```

```
f:                                  1
    PUSH    %14                     2
    MOV     %15,%14                 3
    SUBS    %15,$4,%15              4
@f_body:                            5
    MOV     $3,-4(%14)              6
    ADDS    8(%14),-4(%14),%0       7
    MOV     %0,%13                  8
    JMP     @f_exit                 9
@f_exit:                            10
    MOV     %14,%15                 11
    POP     %14                     12
    RET                             13
...
    PUSH    $7                      21
    CALL    f                       22
    ADDS    %15,$4,%15              23
    MOV     %13,-4(%14)             24
```

7	
addr 22	SP
stari FP	FP, ~~SP~~
3	~~SP~~



$13	10
$14	FP stari
$15	SP

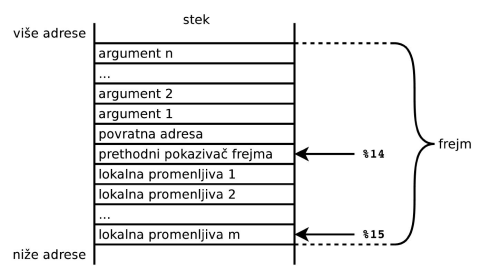
POP - preuzimanje vrednosti sa steka i smeštanje u operand

```c
int f(int p) {
    int a;
    a = 3;
    return p + a;
}

int main() {
    int a;
    a = f(7);
}
```

```
f:                                  1
    PUSH    %14                     2
    MOV     %15,%14                 3
    SUBS    %15,$4,%15              4
@f_body:                            5
    MOV     $3,-4(%14)              6
    ADDS    8(%14),-4(%14),%0       7
    MOV     %0,%13                  8
    JMP     @f_exit                 9
@f_exit:                            10
    MOV     %14,%15                 11
    POP     %14                     12
    RET                             13
...
    PUSH    $7                      21
    CALL    f                       22
    ADDS    %15,$4,%15              23
    MOV     %13,-4(%14)             24
```

7	SP
addr 22 → PC	~~SP~~
stari FP	FP, ~~SP~~
3	~~SP~~



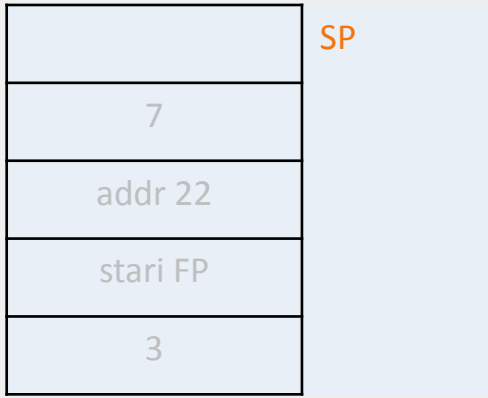
$13	10
$14	FP stari
$15	SP

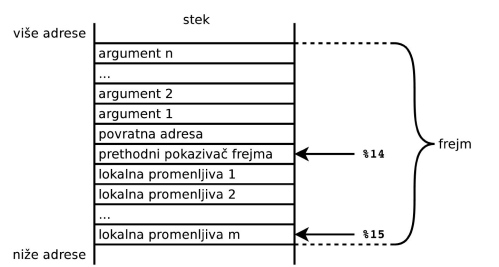
POP - preuzimanje vrednosti sa steka i smeštanje u operand

```
int f(int p) {
   int a;
   a = 3;
   return p + a;
}

int main() {
   int a;
   a = f(7);
}
```

```
f:                                  1
   PUSH    %14                      2
   MOV     %15,%14                  3
   SUBS    %15,$4,%15               4
@f_body:                            5
   MOV     $3,-4(%14)               6
   ADDS    8(%14),-4(%14),%0        7
   MOV     %0,%13                   8
   JMP     @f_exit                  9
@f_exit:                            10
   MOV     %14,%15                  11
   POP     %14                      12
   RET                              13
...
   PUSH    $7                       21
   CALL    f                        22
   ADDS    %15,$4,%15               23
   MOV     %13,-4(%14)              24
```



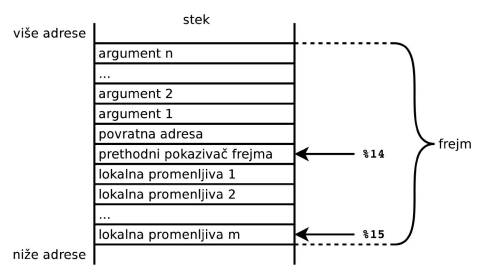


$13	10
$14	FP
$15	SP

```
int f(int p) {
   int a;
   a = 3;
   return p + a;
}

int main() {
   int a;
   a = f(7);
}
```

```
f:                                  1
   PUSH   %14                       2
   MOV    %15,%14                   3
   SUBS   %15,$4,%15                4
@f_body:                            5
   MOV    $3,-4(%14)                6
   ADDS   8(%14),-4(%14),%0         7
   MOV    %0,%13                    8
   JMP    @f_exit                   9
@f_exit:                            10
   MOV    %14,%15                   11
   POP    %14                       12
   RET                              13
...
   PUSH   $7                        21
   CALL   f                         22
   ADDS   %15,$4,%15                23
   MOV    %13,-4(%14)               24
```



$13	~~10~~
$14	FP
$15	SP

```c
int f(int p) {
   int a;
   a = 3;
   return p + a;
}

int main() {
   int a;
   a = f(7);
}
```

```
f:                                  1
   PUSH    %14                      2
   MOV     %15,%14                  3
   SUBS    %15,$4,%15               4
@f_body:                            5
   MOV     $3,-4(%14)               6
   ADDS    8(%14),-4(%14),%0        7
   MOV     %0,%13                   8
   JMP     @f_exit                  9
@f_exit:                            10
   MOV     %14,%15                  11
   POP     %14                      12
   RET                              13
...
   PUSH    $7                       21
   CALL    f                        22
   ADDS    %15,$4,%15               23
   MOV     %13,-4(%14)              24
```

```
int f(int p) {
   int a;
   a = 3;
   return p + a;
}

int main() {
   int a;
   a = f(7);
}
```

```
f:                                  1
   PUSH    %14                      2
   MOV     %15,%14                  3
   SUBS    %15,$4,%15               4
@f_body:                            5
   MOV     $3,-4(%14)               6
   ADDS    8(%14),-4(%14),%0        7
   MOV     %0,%13                   8
   JMP     @f_exit                  9
@f_exit:                            10
   MOV     %14,%15                  11
   POP     %14                      12
   RET                              13
...
   PUSH    $7                       21
   CALL    f                        22
   ADDS    %15,$4,%15               23
   MOV     %13,-4(%14)              24
```

Šta je promenljivo?

```
if(a < b)
   a = 1;
else
   a = 2;
```

```
@if0:
   CMPS   -4(%14),-8(%14)        1
   JGES   @false0                2
@true0:                          3
   MOV    $1,-4(%14)             4
   JMP    @exit0                 5
@false0     :                    6
   MOV    $2,-4(%14)             7
@exit0:                          8
```

```
if(a < b)
   a = 1;
else
   a = 2;
```

```
@if0:
   CMPS   -4(%14),-8(%14)        1
   JGES   @false0                2
@true0:                          3
   MOV    $1,-4(%14)             4
   JMP    @exit0                 5
@false0     :                    6
   MOV    $2,-4(%14)             7
@exit0:                          8
```

- Sličan princip za sve uslovne iskaze, pa time i petlje
- Obratiti pažnju: suprotno ili negacija od (a < b) je (a >= b), a ne (a > b)